Benchmarking LLM-Based Agents for GitHub Issue Enhancement

Pouya Fathollahzadeh, *Student Member, IEEE,* and Ying Zou, *Member, IEEE*

***Abstract*—GitHub issues serve as the primary communication channel between users and developers for reporting bugs, requesting features, and proposing refactoring tasks. Issue descriptions are frequently vague, incomplete, or lacking essential technical details, which hinders both human developers and automated issue-solving agents. Large Language Model (LLM)-based agents offer the potential to automatically *enhance* issue descriptions by adding context, clarifying ambiguities, and structuring information. If agents solve issues from text, better text should mean better fixes. We test this assumption directly and at scale. We construct a SWE-bench-Live–style benchmark of 3,285 issue instances from 613 popular Python repositories and, from it, a validated execution subset of 382 issues, each with a buildable Docker environment and a gold patch. On this subset we run a full enhancer $\times$ solver matrix: coding agents used as *enhancers* (OpenHands, SWE-agent, Aider), a reward-gated selective enhancer (CL-Enhanced), and a no-enhancement baseline, against the same agents used as *solvers* — all driven by a single open-weight model (Qwen3-32B) to isolate the enhancement variable. We evaluate not "did the patch change" but "does the patch pass the repository's tests", using a gold-patch-gated, multi-method evaluability protocol that yields 279 of 382 issues (73.0%) as trustworthy to evaluate. Our results show that: (1) across every enhancer $\times$ solver pairing, issue enhancement produces no statistically significant improvement in fix correctness (paired McNemar exact test, all $p \geq 0.489$); the one promising small-sample signal weakened rather than strengthened as the sample grew, the signature of a non-effect; solver choice, not the wording of the report, dominates outcome (Aider 44% $\gg$ OpenHands 22% $\gg$ SWE-agent 6%). (2) No feature — across an 81-feature SE representation plus 500 TF-IDF terms — predicts a better enhancement under either an intrinsic (reward-model) or a downstream definition of success (all models at chance, ROC-AUC $\approx$ 0.49–0.56); strikingly, enhancement *does* raise the intrinsic quality score (57% of reports improve) yet yields no downstream correctness gain, so text-quality improvements do not transfer to program repair. (3) A qualitative analysis characterizes how the agents rewrite — a uniform Problem/Reproduction/Expected template that lifts the quality score but, being often unfounded, does not help the solver. We release the benchmark, the evaluation pipeline, and all experimental artifacts.**

***Index Terms*—Software Engineering, GitHub Issues, Issue Enhancement, LLM-Based Agents, SWE-bench, Automated Program Repair, Benchmark, Negative Results.**

I. Introduction

GitHub issues are the primary mechanism through which developers and users communicate about software defects, feature requests, and maintenance tasks [2], [1]. A well-written issue report contains a clear problem description, reproduction steps, expected versus actual behavior, and relevant technical context such as version information, stack traces, or code snippets. When these elements are present, both human developers and automated solving agents can effectively diagnose and resolve the underlying problem.

In practice, however, issue descriptions are frequently incomplete, vague, or poorly structured. Prior work on bug report quality has shown that missing reproduction steps, ambiguous descriptions, and insufficient context are among the most common reasons for delayed or failed issue resolution [7], [8]. This quality gap is particularly consequential for automated issue-solving systems, which rely on the issue description as their sole input for generating code patches [1], [3].

The emergence of LLM-based coding agents—such as OpenHands [4], SWE-agent [3], Aider [5], and AutoCodeRover [6]—has opened a new possibility: using these agents not only to *solve* issues but also to *enhance* issue descriptions before solving. An enhancement agent can analyze the codebase, identify relevant source files, extract failing test information, and restructure the issue description to provide a clearer specification. If effective, this pre-processing step could improve the success rate of downstream solving agents. This hypothesis motivates a growing line of issue-enhancement tools, including reward-gated report rewriting. Yet enhancement is almost always evaluated *intrinsically*—does the rewritten report score higher on a text-quality metric—and rarely *extrinsically*: does a better report actually cause a better downstream fix.

This paper closes that gap. Existing benchmarks such as SWE-bench [1] and SWE-bench-Live [2] focus exclusively on issue *solving*, treating the issue description as a fixed input. We instead treat the issue text as a *manipulable* stage. Our approach is built on the *solving-as-evaluation* paradigm: we measure the quality of an enhancement not through subjective text ratings but through its downstream effect on solver performance. Formally, for an enhancement agent E and a solver agent S, the enhancement value is:

$$\Delta_{\text{resolved}} = \text{Resolved}(S, E(\text{issue})) - \text{Resolved}(S, \text{issue}) \quad (1)$$

where a positive Δ_{resolved} indicates that enhancement improved solving performance. This indirect evaluation provides an objective, reproducible measure of enhancement quality that is grounded in functional correctness.

Our study analyzes GitHub issue enhancement along three research questions, each with its own method:

RQ1: What is the ability of generative AI in enhancing GitHub issue reports? We benchmark an enhancer–solver

Pouya Fathollahzadeh and Ying Zou are with the Department of Electrical and Computer Engineering, Queen's University, Kingston, ON K7L 3N6, Canada. E-mail: {pouya.fathollahzadeh, ying.zou}@queensu.ca.

agentic workflow: each enhancer rewrites the report, each solver then attempts a patch, and we measure enhancement ability by whether the resulting patch passes the repository's tests. We evaluate a no-enhancement baseline plus enhancers of increasing sophistication across three solver agents, holding the model fixed.

RQ2: What features are related to a better enhancement? We fit a logistic regression over textual, structural, and enhancement-derived features to identify which are associated with a successful enhancement outcome, under both an intrinsic (reward-model) and an extrinsic (downstream-fix) definition of success.

RQ3: How are GitHub issue reports enhanced by generative-AI agents? We perform a qualitative analysis of the agent-generated reports to characterize recurring enhancement patterns and the failure modes that explain the RQ1 result.

The main contributions of this paper are as follows:

1) We construct a large-scale dataset of 3,285 GitHub issue instances from 613 repositories, and a validated execution subset of 382 instances each with a buildable environment and gold patch, suitable for the extrinsic evaluation of issue enhancement.
2) We propose and apply the *solving-as-evaluation* framework, measuring enhancement quality through downstream solver performance under a controlled model rather than subjective text assessment.
3) We provide a large-scale extrinsic benchmark (RQ1) whose finding is a **rigorous negative result**: no enhancement condition reliably improves fix correctness at any tested scale, and the one small-sample signal is traced to sampling noise by replication.
4) We provide a feature-level (RQ2) and qualitative (RQ3) account of which report/enhancement properties and rewrite behaviors relate to enhancement outcomes, surfacing an intrinsic-improves / extrinsic-flat gap.
5) We contribute an evaluation methodology—a gold-patch-gated, three-method evaluability gate (73.0% coverage, 279/382) with an artifact-recovery discipline—that we argue is a prerequisite for trustworthy agent-repair measurement.
6) We release the benchmark dataset, evaluation pipeline, and all experimental artifacts in our replication package [15].

The rest of the paper is organized as follows: Section II discusses background and related work. Section III presents our methodology. Section IV presents the results for each research question. Section V discusses implications. Section VI discusses threats to validity. Section VII concludes.

II. Background and Related Work

A. Issue Report Quality

The quality of bug reports and issue descriptions has been extensively studied. Bettenburg et al. [7] surveyed developers and found that steps to reproduce, stack traces, and test cases are the most valuable elements in bug reports, yet they are frequently missing. Zimmermann et al. [8] identified that incomplete information is the primary barrier to bug fixing, with a large fraction of reports lacking sufficient detail.

Several approaches have been proposed to improve bug report quality. Duplicate detection systems [9], [10] help reduce redundant reports. Automated triaging [11], [12] routes reports to appropriate developers. More recently, approaches that automatically augment bug reports with relevant source code [13], [14] have shown promise. All of this measures quality *intrinsically* (a text or model score). To our knowledge no prior study measures whether such rewriting propagates to downstream *repair correctness*, which is the gap we close. Our work also differs by leveraging LLM-based agents as enhancement tools rather than rule-based or retrieval-based augmentation.

B. SWE-bench and Issue-Solving Benchmarks

SWE-bench [1] introduced a benchmark of software engineering problems drawn from real GitHub issues. Given a codebase and an issue description, a model generates a patch evaluated against unit tests. The benchmark uses two test categories: *fail-to-pass* (F2P) tests that should change from failing to passing after the patch, and *pass-to-pass* (P2P) tests that must continue passing to ensure no regressions. SWE-bench-Live [2] extended this paradigm with an automatically updatable benchmark spanning many repositories and tasks derived from recent issues, addressing staleness and limited coverage. These systems are evaluated on the *fix*; the issue text is taken as fixed input. We reuse the solving step as an evaluation instrument and ask whether *improving* the issue text changes the fix—an extrinsic question these benchmarks do not pose.

C. LLM-Based Coding Agents

LLM-based coding agents form a diverse ecosystem of tools capable of interacting with codebases, executing commands, and generating patches. SWE-agent [3] introduces an agent-computer interface designed for software engineering tasks. OpenHands [4] provides a platform for AI-powered software development agents. Aider [5] is a command-line tool for AI-assisted pair programming. AutoCodeRover [6] performs autonomous program improvement through structured code search. While designed for solving, their ability to understand code context, identify relevant files, and generate structured analysis makes them natural candidates for issue enhancement—a use case we systematically evaluate.

D. Issue Enhancement

Issue enhancement refers to automatically improving an issue description by adding relevant context, clarifying ambiguities, restructuring content, or appending diagnostic information. Unlike issue solving, which produces a code patch, enhancement produces an improved textual description that can be consumed by either human developers or automated solvers. To the best of our knowledge, no prior work has systematically benchmarked issue enhancement agents or measured the downstream solving impact of enhancement. Our work fills this gap by providing both a benchmark and a comprehensive extrinsic evaluation.

III. METHODOLOGY

Fig. 1 presents an overview of our approach, which consists of two components: (1) a data collection and dataset construction pipeline, and (2) an enhancement–solving–evaluation workflow.

A. Data Collection and Dataset Construction

We construct a SWE-bench-style benchmark from recent GitHub issues to ensure evaluation data postdates the training cutoffs of the LLMs under study. Fig. 2 illustrates the pipeline.

1) Stage 1: Repository Selection: We fetch GitHub repositories with more than 1,000 stars, yielding 10,609 repositories, and apply three quality filters: **(i) language**—Python constitutes more than 60% of the codebase; **(ii) fork activity**—more than 200 forks; and **(iii) issue and PR volume**—the combined count exceeds 200. After these filters, 3,621 repositories with 10,463 associated issues remain.

2) Stage 2: Temporal Filtering: To ensure recency and avoid training-data contamination, only issues created after August 1, 2025 are retained. This reduces the dataset to 2,748 repositories with 7,714 issues.

3) Stage 3: Merge-Based Linking and Test Coverage: We retain only issues that (a) were closed by a *merged* pull request (establishing a ground-truth fix); (b) whose PR includes a non-empty code patch and a non-empty test patch; and (c) that have at least one pass-to-pass test (P2P > 0). This yields 613 repositories with 3,285 issues.

4) Stage 4: Validated Execution Environments: For the extrinsic, execution-based evaluation, each issue must have a reproducible environment in which tests actually run. We build a per-issue Docker image and verify that the repository's designated tests execute and that the gold patch is applicable. The instances that pass this validation on our primary compute node form the **382-issue execution subset** used for the enhancer $\times$ solver matrix. Each such instance carries a buildable image, a recorded test command, and a gold patch.

B. Issue Enhancement Conditions

We evaluate a no-enhancement **baseline** against a ladder of enhancers of increasing sophistication, so that scale of machinery is controlled:

- **No Enhancement (Baseline):** the original issue is passed directly to the solver. Control condition.
- **Zero-Shot (Raw LLM):** a single LLM call restructures the issue without agent-level tool use, measuring the value of prompt-only rewriting.
- **Ready-to-use agents: OpenHands** [4], **SWE-agent** [3], and **Aider** [5], each a $\sim$30-step agent loop given the issue and repository access and instructed to enhance the description. These rewrite *unconditionally*.
- **CL-Enhanced (reward-gated):** a managed enhancer that *gates* on a learned reward model and only rewrites reports it judges low-quality, using retrieval-augmented (RAG) grounding. This is a *selective* enhancer (Section V-C).

C. Solver Agents

To evaluate enhancement through downstream solving impact, we use three independent solver agents: **OpenHands** [4], **SWE-agent** [3], and **Aider** [5], each in solving mode. Using multiple independent solvers reduces the risk that enhancement effects are artifacts of a single solver; an enhancement that helps across solvers provides stronger evidence of genuine quality improvement.

Controlled model. Every enhancer and solver is driven by a single open-weight model, **Qwen3-32B**, served from a private inference endpoint at temperature 0.0. Holding the model fixed isolates the *enhancement method* as the only variable, so any difference is attributable to the rewriting, not to model capability. Enhancement and solving execute in isolated Docker environments to prevent cross-contamination.

D. Evaluation Framework

Our evaluation uses the SWE-bench-Live harness [2], [1] to assess solver-generated patches against ground-truth tests.

1) Resolved Rate (P2P): We score a patch as *resolved* if, after applying it (plus the issue's test patch) inside the validated image, the repository's designated P2P tests execute and pass with no failures:

$$\text{Resolved} = \begin{cases} 1 & \text{if all P2P tests pass} \wedge |\text{P2P}_{\text{observed}}| > 0 \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

We report **P2P-resolved** rather than the stricter SWE-bench F2P criterion because the offline-derived F2P labels proved unusable at this scale—only 4 of 74 evaluable instances carried a gold-passing F2P label—so requiring them would collapse the sample. P2P-resolved is a no-regression + de-facto-fix signal (the applied test patch supplies the fix-relevant tests); we treat it as directional, and Section VI discusses this threat.

2) The Evaluability Gate: Not every issue can be trusted to evaluate. We gate on a **gold-patch probe**: an instance is *evaluable* only if the gold patch itself passes under the chosen method. Because a single generic test command fails on many repositories (framework-specific runners, stale node-IDs), we combine three methods per instance and keep whichever the gold patch validates: **v2**, the repository's real validated test commands recovered from the build; **v3**, a label-agnostic file-level run (execute the test files, require ≥ 1 passed and 0 failed—robust to mislabeled node-IDs); and **v1**, a generic fallback. This lifted evaluable coverage to **73.0% overall (279/382)**—and up to 75.8% on the largest tranche—versus $\sim$30% for a single generic method.

3) Enhancement Coverage and Delta: Coverage is the proportion of issues an enhancer actually modifies, Coverage $= |\{i : E(i) \neq i\}|/|I|$. The primary comparison metric is the resolved-count *delta* against the baseline (Eq. 1); a positive delta indicates net improvement, a negative delta indicates that enhancement *harmed* solving.

4) Statistics: For each enhancer-vs-baseline comparison on a given solver, outcomes are **paired** (same issues, with vs. without enhancement), so we use the **exact McNemar test** on the discordant pairs and report the pair counts. We report

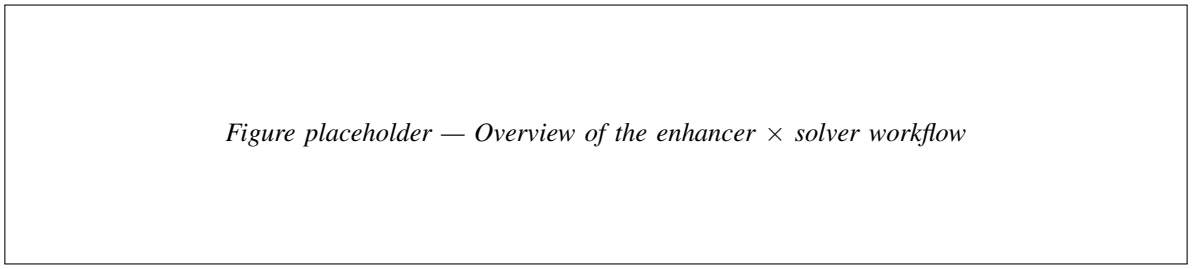

Fig. 1. Overview of our approach. Original issues are processed by each enhancement condition (including a no-enhancement baseline). Each enhanced issue is then solved by three independent solver agents. Enhancement quality is measured through the change in solver success rate (Resolved / P2P metrics).

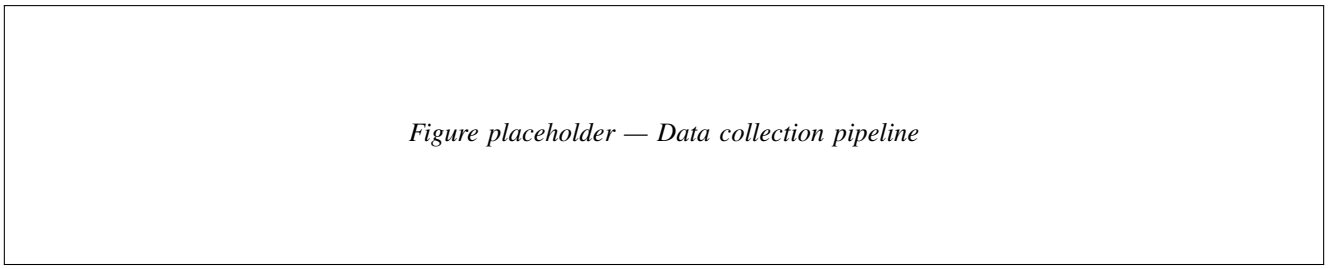

Fig. 2. Data collection pipeline. Starting from GitHub repositories with more than 1,000 stars, we apply successive filters for language composition, activity, recency, and merge-based linking to produce 3,285 benchmark-ready instances across 613 repositories; validated environment construction then yields the 382-instance execution subset used for the enhancer × solver matrix.

TABLE I
RQ1 CORRECTNESS MATRIX AT $n = 382$: RESOLVED / 279 EVALUABLE (P2P), WITH Δ VS. BASELINE AND MCNEMAR EXACT p. ROWS ARE SOLVERS, COLUMNS ENHANCERS.

Solver (baseline)	enh: OpenHands	enh: SWE-agent	enh: Aider
OpenHands (60, 22%)	58 (−2, .910)	66 (+6, .561)	67 (+7, .489)
SWE-agent (18, 6%)	18 (0, 1.0)	19 (+1, 1.0)	19 (+1, 1.0)
Aider (124, 44%)	125 (+1, 1.0)	119 (−5, .649)	124 (0, 1.0)

TABLE II
OPENHANDS-ENHANCEMENT → AIDER SOLVER, ACROSS SCALES.

Scale	Resolved (base → enh)	Δ	McNemar p
$n = 100$ (74 eval)	30 → 34	+4	0.388
$n = 200$ (141 eval)	64 → 66	+2	0.856
$n = 382$ (279 eval)	124 → 125	+1	1.000

Wilson 95% confidence intervals on each resolved rate. Effect sizes are absolute differences in resolved count over the fixed evaluable denominator (empty or failed patches count as unresolved).

IV. RESULTS

A. RQ1: What Is the Ability of Generative AI in Enhancing Issue Reports?

1) The correctness matrix — no enhancer improves any solver: Table I gives the final correctness matrix at $n = 382$ (279 gold-evaluable issues), with each enhancer compared to the baseline for each solver. **No enhancer × solver comparison reaches significance; every $p \geq 0.489$, and the largest absolute effect is 7 issues out of 279.** The one comparison that could be called a "best case"—Aider-enhancement into the OpenHands solver (+7)—is nowhere near significance and is not the pairing any prior signal predicted.

2) The one promising signal was sampling noise: At $n = 100$ the headline pairing (OpenHands-enhancement → Aider solver) looked promising: +4 resolved (30→34), $p = 0.388$. Replication kills it (Table II): the effect **shrank toward zero and the p-value rose toward 1** as data accumulated—the diagnostic of a non-effect, not of insufficient power. A genuine effect of fixed size sharpens under more data; this did the opposite. The same pattern held for the OpenHands solver, where an $n = 74$ "+6" reversed to −3 on a fresh 100 issues before settling near null. Had we stopped at $n = 100$, the paper would have carried a false positive.

3) Solver choice dominates; enhancement perturbs without direction: Two robust facts survive at every scale. **(1) Solver capability is the dominant factor:** baseline resolved rates are Aider 44% ≫ OpenHands 22% ≫ SWE-agent 6%, stable across all three tranches and far larger than any enhancement effect—the choice of *who fixes* matters an order of magnitude more than *how the report is worded*. **(2) Enhancement is active but directionless:** for the two responsive solvers, 65–78 issues per comparison change outcome (discordant pairs)—23–28% of the evaluable set—but the flips are balanced (helps ≈ hurts), so there is no net gain. A model-fit note reinforces the point: the SWE-agent solver is effectively inert to enhancement (0–1 discordant pairs) because, with Qwen3-32B, the large majority of its runs terminate in an agent-protocol error before any patch is produced (∼86% in a 100-issue audit)—a failure of the model–harness interface, not of the issue text.

TABLE III
TOP SE FEATURES *added* BY ENHANCEMENT (ABSENT IN THE ORIGINAL, PRESENT AFTER; OPENHANDS, $N = 304$ TRULY-ENHANCED).

Feature added	Issues gained	% of enhanced
reproduction steps	259	**85%**
expected behavior	138	45%
list items	135	44%
actual behavior	124	41%
section headers	121	40%
logs	66	22%
inline code	60	20%
environment info	26	9%
links	15	5%
checkboxes	12	4%

RQ1 Summary. Across the full enhancer $\times$ solver matrix on 279 evaluable issues, no enhancer improves fix correctness for any solver (all McNemar $p \geq 0.489$). The one small-sample signal regressed toward zero under replication—a non-effect. Solver capability (Aider 44% $\gg$ OpenHands 22% $\gg$ SWE-agent 6%) dominates, and enhancement perturbs a quarter-to-a-third of outcomes without net direction.

B. RQ2: What Features Are Related to a Better Enhancement?

Approach. From each issue's title and body we extract the full **81-feature SE representation** of prior reward-model work—spanning size, readability (Flesch, Flesch–Kincaid, ARI), part-of-speech counts, structural markup (fenced/inline code, URLs, images, checklists, headers, lists, tables), content flags (stack trace, error message, logs, reproduction steps, expected/actual behavior, environment info), and lexical statistics. We use it two ways: *descriptively*, tallying which features an enhancement *adds*; and *predictively*, regressing a success outcome on the features under two operationalizations—an **extrinsic** label (the enhanced report yields a downstream resolved patch), over the 81 SE features; and an **intrinsic** label (the learned reward model scores the enhanced report ≥ 0.5), over the full **581-feature** set (81 SE + 500 TF-IDF terms). Both use L1-regularized logistic regression tuned by 5-fold cross-validation.

1) What enhancement adds: Comparing SE features before and after enhancement (OpenHands, over its 304 truly-enhanced issues), the additions are a **uniform template**: the agent rewrites almost every report into a structured Problem / Reproduction / Expected / Actual form (Table III). These are precisely the fields the reward model rewards, which explains the intrinsic gain below; but several—reproduction steps, expected/actual behavior—are facts the agent cannot know and often *invents*, foreshadowing why they do not help the solver.

2) No feature predicts a better enhancement: **Extrinsic view.** The 81-feature model reaches 5-fold cross-validated ROC-AUC $= 0.55$—indistinguishable from the 0.50 no-signal line. L1 retains 14 features with modest odds ratios (Table IV); the strongest are *informativeness* properties of the *original* report (bodies with logs, OR 1.36; a stack trace, OR 1.20), not of the rewrite. On the 65 issues where enhancement *changed* the Aider outcome, it helped 33 and hurt 32; predicting helped-vs-hurt yields ROC-AUC ≈ 0.58 at $n = 65$—no reliable signal.

TABLE IV
L1-SELECTED SE FEATURES FOR A RESOLVED FIX (BEST CELL, $n = 279$; FULL 81-FEATURE MODEL, STANDARDIZED OR). REMAINING SELECTED FEATURES HAVE $|\text{OR} - 1| < 0.05$; MODEL AUC $= 0.55$.

Feature (from title/body)	Standardized OR
`body_has_logs`	1.36
`body_has_stack_trace`	1.20
`body_has_environment_info`	1.09
`body_flesch_reading_ease`	1.03
`title_contains_feature`	0.87
`num_urls_in_body`	0.87
`body_avg_word_length`	0.88
`title_ends_with_punctuation`	0.89
`title_stopword_ratio`	0.90

TABLE V
RQ2 — ALL THREE FEATURE MODELS LAND AT CHANCE.

Model	Features	Outcome	n	Pos.	CV AUC
extrinsic	81 SE	downstream resolved	279	45%	**0.55**
intrinsic	581	reward ≥ 0.5	382	69%	**0.49**
direction	81 SE	helped vs. hurt	65	51%	**0.58**

Intrinsic view — the key contrast. With the full 581-feature representation and the reward model, enhancement is *not* inert: original score mean $0.473 \rightarrow$ enhanced 0.578; $218/382$ (57%) of reports improve and 122 (32%) cross from low- to high-quality. Yet *which* issue gets a successful enhancement is still unpredictable from its 581 features (AUC $= 0.49$, chance). The agent enhances **uniformly**—turning almost any report into a similarly structured, higher-scoring one—so no input feature discriminates success. Table V summarizes: all three feature models land at chance.

RQ2 Summary. No feature—across the full 81-SE + 500-TF-IDF representation—is meaningfully associated with a better enhancement, on either an intrinsic or a downstream definition. The sharpest finding is the contrast: enhancement reliably raises the *intrinsic* quality score (57% improve, 32% cross threshold) but produces **no downstream correctness gain** and no feature-predictable subpopulation. Text-quality gains do not transfer to program repair.

C. RQ3: How Are Issue Reports Enhanced?

(Qualitative analysis in progress; corpus collected.) We open-code a stratified sample of agent-generated enhancements (across conditions, sampling successes, harms, and no-change cases) for (a) the **enhancement patterns** applied—restructuring into Problem / Reproduction / Expected sections, appending root-cause hypotheses, extracting stack traces, adding code context—and (b) the **failure modes**—hallucinated details, over-specification, loss of the original signal, or (for the reward-gated agent) abstention. Two coders label independently; we report inter-rater agreement (Cohen's κ) and a pattern $\times$ outcome cross-tabulation linking each rewrite move to whether it helped, hurt, or left the fix unchanged. The

RQ2 feature-delta analysis (Table III) already indicates the mechanism: the agents apply a *uniform* template that lifts the quality score but, being often unfounded, does not help the solver.

RQ3 Summary (preliminary). Enhancement is a uniform restructuring into Problem/Reproduction/Expected/Actual sections. Because reproduction and expected/actual fields are frequently invented rather than grounded, they raise intrinsic quality without adding solver-actionable signal—the mechanism behind the directionless perturbation in RQ1.

V. Discussion

A. Why Enhancement Fails to Transfer

The issues in an execution-gated repair benchmark are, by construction, *already actionable*—they were selected because a solver could plausibly fix them. Rewriting an already-actionable report adds little signal and can just as easily perturb the solver off a working trajectory as onto one. Enhancement may help most exactly where these benchmarks contain the fewest examples: genuinely underspecified reports. This reframes the "enhancement paradox"—that adding information does not help—as a selection effect of the benchmark, not a universal law.

B. Measurement Discipline Was Decisive

Two classes of infrastructure artifact would each have manufactured a false result if left uncorrected, and both were caught by cross-checking rather than trusting raw counts: (i) **solver timeouts under concurrency**—slow iterative solvers, and even Aider on heavy repositories, hit wall-clock limits and produced empty patches that are *not* real failures; re-solving at lower concurrency recovered them (e.g., 20 of 27 Aider timeouts recovered at a median 881 s). (ii) **Permission-lost patches**—the runtime occasionally wrote its patch file unreadable to the host, silently zeroing $\sim$2% of OpenHands results; these were recovered by a privileged read. A study scoring the raw output would have reported spuriously low—and unevenly distributed—enhancement effects.

C. The CL-Enhanced (Reward-Gated) Arm

The agents above rewrite **unconditionally**. Our managed enhancer, CL-Enhanced, instead *gates* on a learned reward model and only rewrites reports it judges low-quality. Its reward model judged **216 of 382 issues (57%) already adequate and abstained** (returning the original), acting on the other **166 (43%)** with RAG-grounded rewriting—a *selective* enhancer and a cleaner test of whether *targeted* enhancement transfers. The methodologically correct comparison is on the subset it *chose to rewrite*; on the **129 evaluable issues CL-Enhanced rewrote, no solver improves** (Table VI). A cautionary note: over *all* 279 evaluable, the SWE-agent comparison reads $\Delta{-}9$, $p = 0.004$—*apparently* significant. It is an artifact of non-determinism: 6 of those 9 "losses" fall on *abstained* issues whose text was identical to baseline, so they reflect run-to-run solver variance, not any effect of rewriting. The enhanced-only subset removes this confound and shows the true null. The takeaway completes the arc: **neither unconditional generic enhancement nor selective reward-gated enhancement improves downstream fix correctness.**

TABLE VI
CL-Enhanced vs. baseline, on the 129 evaluable issues it rewrote (McNemar exact).

Solver	Baseline $\rightarrow$ CL-Enhanced	Δ	p
OpenHands	$32 \rightarrow 32$	0	1.00
SWE-agent	$8 \rightarrow 5$	-3	0.25
Aider	$58 \rightarrow 55$	-3	0.74

D. Implications for Practice

For automated repair on already-actionable issues, effort is better spent on the solver and the underlying model than on rewriting the input. Enhancement systems should prioritize verified, factual context injection over generative analysis, and enhancer–solver co-design over generic rewriting. The solving-as-evaluation paradigm gives researchers an objective, reproducible measure of enhancement quality; before deploying enhancement, teams should validate that their specific enhancer–solver combination produces positive deltas on representative, execution-gated data.

VI. Threats to Validity

Construct — the metric is P2P-only. Our resolved criterion is no-regression on P2P tests, augmented by the applied test patch, not the stricter F2P criterion, which the offline labels could not support at scale (only 4 of 74 evaluable instances carried a gold-passing F2P label). Consequently our result rules out a *large* correctness effect and shows no *directional* one, but cannot certify a strict-fix effect of small magnitude; re-deriving fix-tests from execution is the highest-value follow-up.

Internal — run-to-run solver non-determinism. Baseline and enhanced conditions are separate stochastic runs, so a fraction of the discordant pairs reflects variance rather than enhancement. This is quantifiable in the CL-Enhanced arm: on abstained (text-identical) issues, 6 of 9 SWE-agent "losses" are pure variance, inflating the full-set comparison to a spurious $p = 0.004$ that the enhanced-only subset corrects to a null. Because our headline effects are null or small, this variance widens confidence intervals rather than creating a false positive; a definitive study should run multiple seeds per condition.

Construct — CL-Enhanced operates out of distribution. Its reward model and RAG corpus were trained on an issue-*difficulty* dataset, not repository-repair issues; applying it here is a *transfer* test, and its high abstention (57%) is partly an out-of-domain artifact.

External — single model, one collection, execution-gated selection. All conditions use a single open-weight model (Qwen3-32B); a stronger or function-calling-native model could change absolute rates (notably SWE-agent's) and conceivably the enhancement effect. The 382 issues are, by construction, actionable, so they under-represent the genuinely underspecified reports where enhancement is most plausibly

useful. Our dataset is also restricted to popular Python repositories, limiting generalizability.

Conclusion — evaluability coverage. We score only the 73.0% (279/382) gold-validated subset; the un-evaluable 27% are excluded rather than assumed-failed. The multi-method gate is designed to keep that fraction small and repository-diverse.

VII. Conclusion

We asked whether improving a bug report's text improves an LLM agent's fix, and answered it extrinsically, at scale, by executing patches against real tests. Across 382 issues, a baseline and enhancers spanning a zero-shot prompt to a reward-gated agent, three solvers, and a fixed model, **issue enhancement produced no statistically significant improvement in fix correctness** (all McNemar $p \geq 0.489$ at $n = 279$), and the one small-sample signal regressed toward zero under replication. What proved robust is that solver choice governs outcome (Aider 44% $\gg$ OpenHands 22% $\gg$ SWE-agent 6%) and that enhancement perturbs a quarter-to-a-third of outcomes without net direction. The negative result holds for our own selective, reward-gated method as well: on the issues it rewrote it improves no solver, and its most notable behavior is to abstain on the majority of already-actionable reports. No feature predicts a better enhancement, yet enhancement reliably raises the intrinsic quality score—text-quality gains that do not transfer to program repair. We offer this as a rigorous negative result, made trustworthy by a gold-gated, multi-method evaluability protocol (73.0% coverage) and an artifact-recovery discipline. Whether enhancement helps on genuinely *underspecified* reports, and whether a stronger solver model changes the picture, are the natural next studies. We release the benchmark, evaluation pipeline, and experimental artifacts to support future research.

References

[1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. ICLR*, 2024.

[2] L. Zhang, S. He, C. Zhang, Y. Kang, B. Li, C. Xie, J. Wang, M. Wang, Y. Huang, S. Fu, E. Nallipogu, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang, "SWE-bench goes live!" *arXiv preprint arXiv:2505.23419*, 2025.

[3] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Proc. NeurIPS*, 2024.

[4] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zhang, B. Yu, J. Yang, S. Yao, Z. Liu, and G. Neubig, "OpenHands: An open platform for AI software developers as generalist agents," *arXiv preprint arXiv:2407.16741*, 2024.

[5] P. Gauthier, "Aider: AI pair programming in your terminal," 2024. [Online]. Available: https://aider.chat

[6] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "AutoCodeRover: Autonomous program improvement," in *Proc. ISSTA*, 2024, pp. 1592–1604.

[7] N. Bettenburg, S. Just, A. Schröter, C. Weiss, R. Premraj, and T. Zimmermann, "What makes a good bug report?" in *Proc. FSE*, 2008, pp. 308–318.

[8] T. Zimmermann, R. Premraj, N. Bettenburg, S. Just, A. Schröter, and C. Weiss, "What makes a good bug report?" *IEEE Trans. Softw. Eng.*, vol. 36, no. 5, pp. 618–643, 2010.

[9] C. Sun, D. Lo, S. Khoo, and J. Jiang, "Towards more accurate retrieval of duplicate bug reports," in *Proc. ASE*, 2011, pp. 253–262.

[10] A. Lazar, S. Ritchey, and B. Sharif, "Generating duplicate bug report datasets," in *Proc. MSR*, 2014, pp. 392–395.

[11] J. Anvik, L. Hiew, and G. C. Murphy, "Who should fix this bug?" in *Proc. ICSE*, 2006, pp. 361–370.

[12] G. C. Murphy and D. Cubranic, "Automatic bug triage using text categorization," in *Proc. SEKE*, 2004, pp. 92–97.

[13] C. P. Wong, Y. Xiong, H. Zhang, D. Hao, L. Zhang, and H. Mei, "Boosting bug-report-oriented fault localization with segmentation and stack-trace analysis," in *Proc. ICSME*, 2014, pp. 181–190.

[14] L. Moreno, J. J. Treadway, A. Marcus, and W. Shen, "On the use of stack traces to improve text retrieval-based bug localization," in *Proc. ICSME*, 2015, pp. 151–160.

[15] P. Fathollahzadeh and Y. Zou, "Replication package: Benchmarking LLM-based agents for GitHub issue enhancement," 2026. [Online]. Available: https://github.com/pouyafath/BenchmarkLLMAgent